The Journal of Open Source Software

FEniCSWEB - An Open-Source Application for Thermal Conduction Analysis: Integrating FEniCS with Django

Harish Ragul Rajaramaduraikarthik [1,2]

1 JJ College of Engineering and Technology, Trichy, India **2** School of Engineering and Technology, Dhanalakshmi Srinivasan University, Perambalur, India

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 19 January 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Abstract

This `FEniCSWEB` project presents a powerful computation initiative for solving numerical analysis problems (like Finite Element Analysis and Computational Fluid Dynamics) using FEniCS integrated with Django. The application provides a web-based interface for engineers, researchers, and students to define, generate and analyze finite element models of thermal systems. For the initial phase of this work, I created a solver for 1D, 2D, and 3D Steady State Thermal Conduction Problems. The solver module employs the finite element method (FEM) to solve the governing heat conduction equation, incorporating user-defined thermal conductivity, boundary conditions, and heat source terms. Static visualization of both mesh structures and computed temperature distributions is provided using Matplotlib to get detailed insights into the thermal behavior of the system. The application stores all the data like mesh and solutions, which allows users for future reuse and retrieval. The application is designed to be open-source to serve as a foundational tool for educational purposes and research. The combination of FEniCS and Django bridges the gap between advanced numerical analysis and accessibility for non-experts.

Statement of need

The Existing finite element analysis (FEA) softwares (like Ansys, Abaqus) often requires expensive licenses and extensive training. Many Open Source FEA tools have deep learning curves, making them challenging for beginners; it is not so user friendly for non-experts. This project simplifies the process by a web interface, allowing users to easily define problems, set boundary conditions, and visualize results. By leveraging Django and FEniCS, this project combines advanced numerical analysis with web platforms.

Design and Development

For the initial phase this project only focused on the steady state thermal conduction problem. This project development is split into two modules 1. Mesh Generation Module, 2. Solver module.

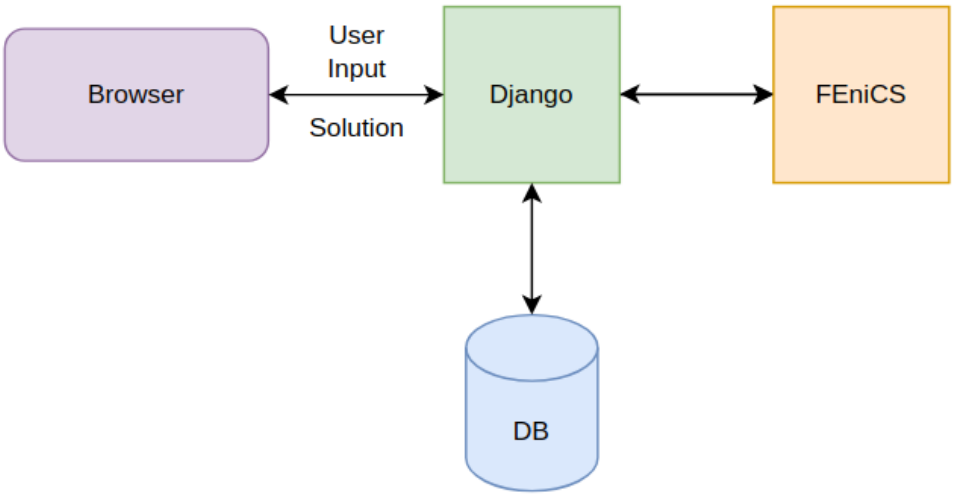
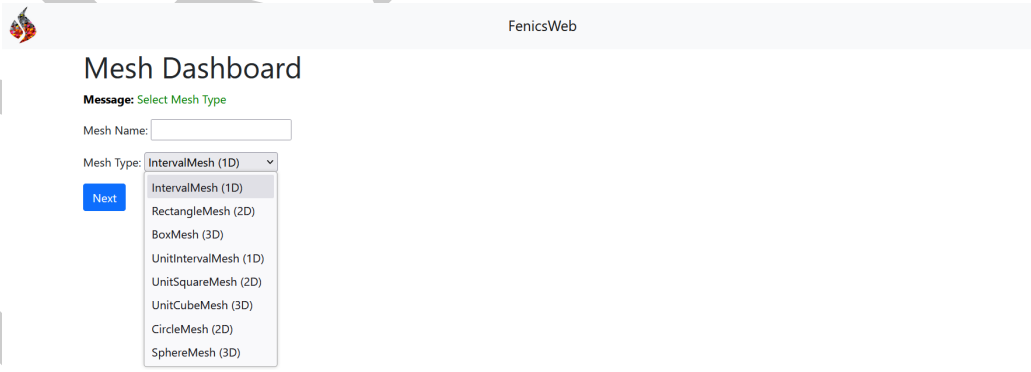
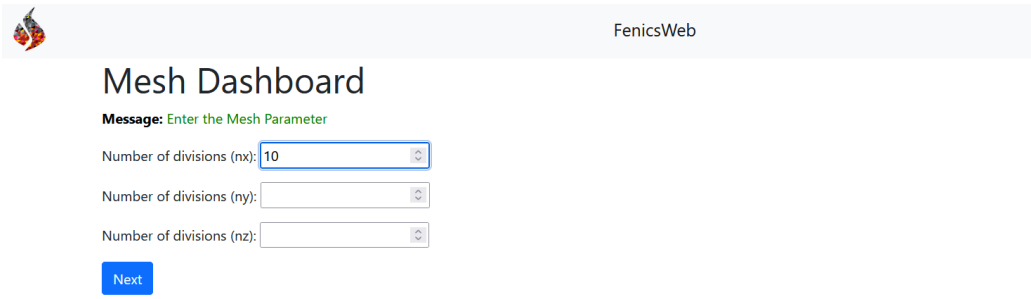


Figure 1: Integration of Fenics with Django.

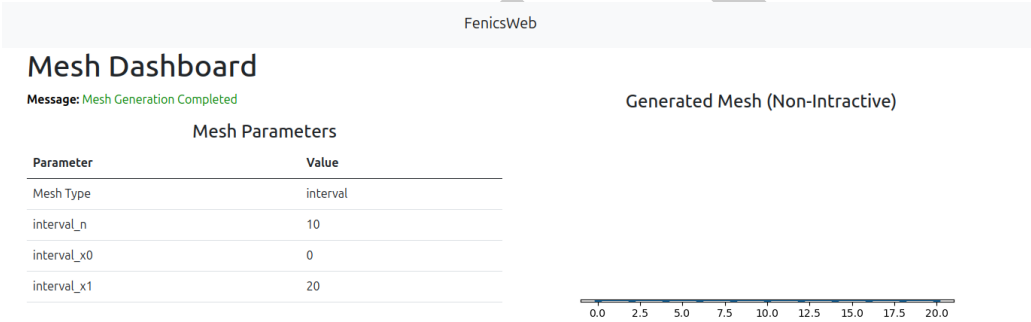
a. Mesh Generation Module:

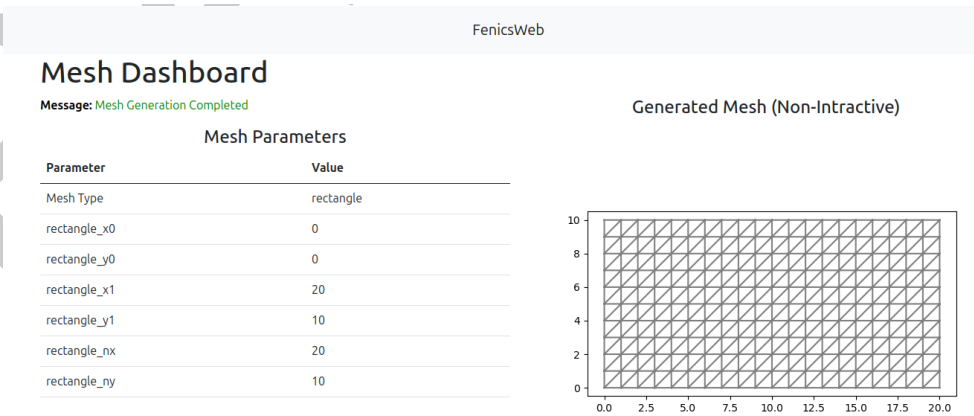
The Django form is created to enable users to define domains (meshes) with various geometries such as intervals, rectangles, and cubes. Use FEniCS for mesh generation based on user input and create visualizations using Matplotlib for plot and the mesh data storage in the database for reuse. The mesh module is routed into `http://127.0.0.1:8000/mesh/` which we can generate meshs like `IntervalMesh (1D)`, `RectangleMesh (2D)`, `BoxMesh (3D)`, `UnitIntervalMesh (1D)`, `UnitSquareMesh (2D)`, `UnitCubeMesh (3D)`



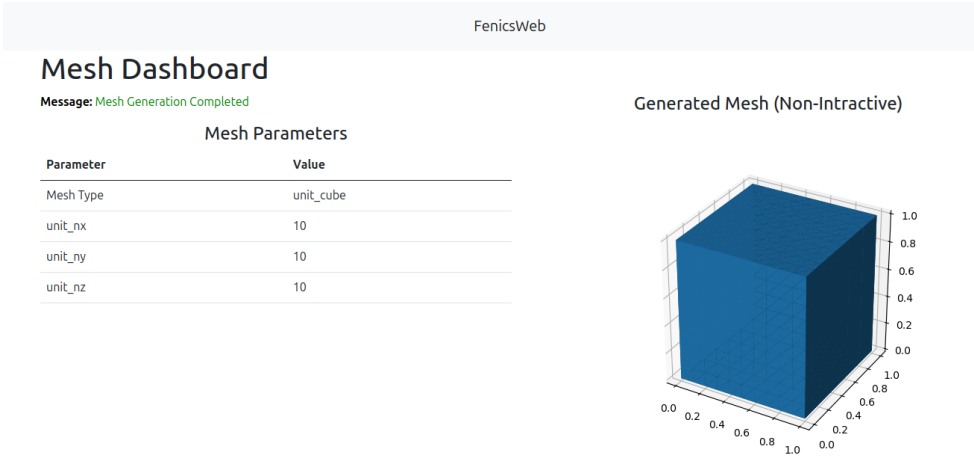


1D Mesh





2D Mesh



3D Mesh

b. Solver Module:

Developed a Python-based solver using FEniCS to compute the temperature field (u) by solving the steady-state heat conduction equation:

$-\nabla\cdot(k\nabla u)=f$ in the domain

$u=u_0$ on Dirichlet boundaries

Forms are implement for users to specify boundary conditions, material properties, and heat sources which assemble and solve the finite element equations and visualization the solution. Visualization using Matplotlib to visualize Generated meshes and their properties, Computed temperature distributions and heat flux vectors. It provides both PNG visualization and detailed result repect to the coordinates.

Problem Statement

The heat equation we solve is:

$-\Delta u = f \quad \text{in } \Omega,$

with the boundary condition:

$u = u_D \quad \text{in } \partial\Omega$

where:

Δu is the Laplace operator (second derivative).

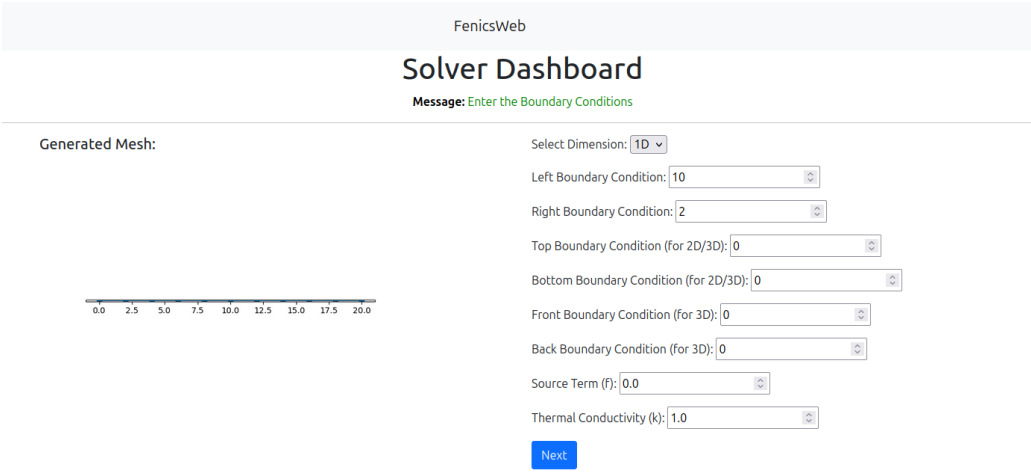
f is the source term.

u_D is the prescribed temperature on the boundary (Ω).

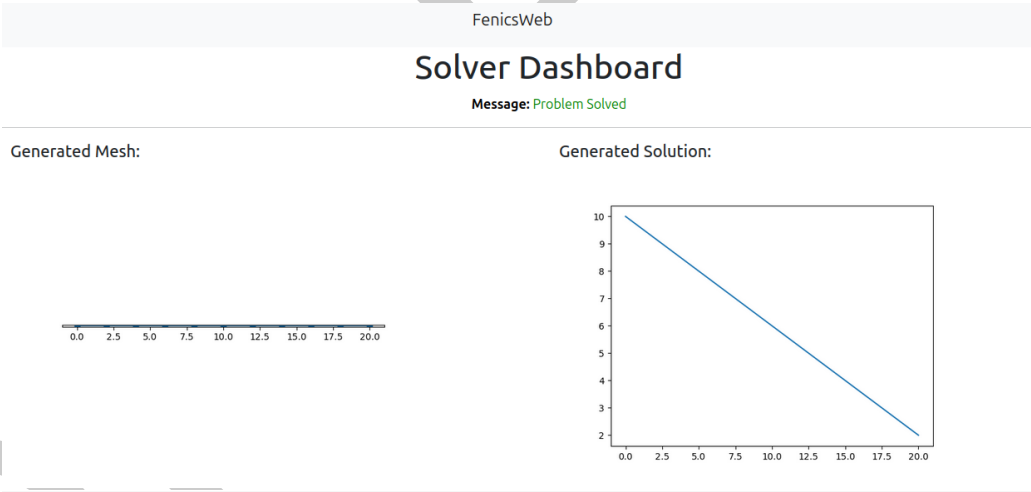
The Steady State Thermal Conduction Solver is routed to `http://127.0.0.1:8000/SteadyStateThermal/`

DRAFT

1D Mesh



For the 1D problem it has only 2 boundaries and zero for the rest of the fields. These boundaries are `Dirichlet boundaries`; the `Neumann boundaries` will be updated in the next version.



Solution Data:

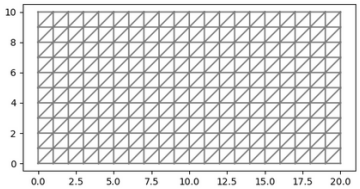
Search for coordinates or s | Download CSV

Coordinates	Solution
[0.0]	10.0
[2.0]	9.2
[4.0]	8.4
[6.0]	7.6000000000000005
[8.0]	6.800000000000002
[10.0]	6.000000000000002
[12.0]	5.200000000000001
[14.0]	4.4
[16.0]	3.6000000000000005
[18.0]	2.8000000000000007
[20.0]	2.0

The Journal of Open Source Software

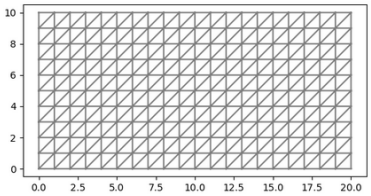
2D Mesh

Generated Mesh:

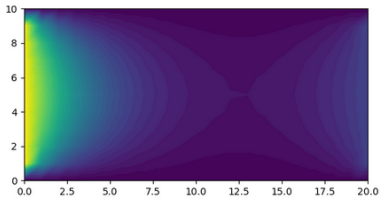


Select Dimension: 2D
Left Boundary Condition: 10
Right Boundary Condition: 2
Top Boundary Condition (for 2D/3D): 0
Bottom Boundary Condition (for 2D/3D): 0
Front Boundary Condition (for 3D): 0
Back Boundary Condition (for 3D): 0
Source Term (f): 0
Thermal Conductivity (k): 1.0
Next

Generated Mesh:

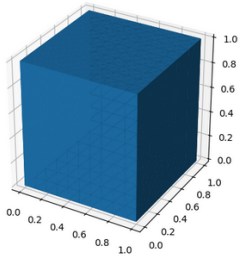


Generated Solution:



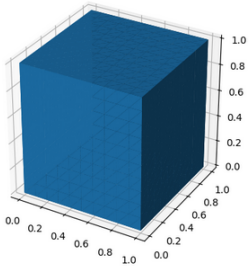
3D Mesh

Generated Mesh:

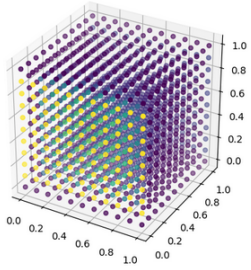


Select Dimension: 3D
Left Boundary Condition: 0
Right Boundary Condition: 0
Top Boundary Condition (for 2D/3D): 2
Bottom Boundary Condition (for 2D/3D): 10
Front Boundary Condition (for 3D): 0
Back Boundary Condition (for 3D): 0
Source Term (f): 0.0
Thermal Conductivity (k): 1.0
Next

Generated Mesh:



Generated Solution:



Note

- For the boundary value temperature refers with the unit of `°C` Celsius then use `(W/m-°C)` watts per meter per degree Celsius
- For the boundary value temperature refers with the unit of `K` Kelvin then use `(W/m·K)` watts per meter Kelvin
- For the insulated boundary the Neumann Boundary Condition will be added in next phase of the work

Future Work

- Implementation of custom domain (Mesh)
- Implementation of other thermal problems (Convection and Radiation) and boundaries (Neumann and Robin)

References

- Mats G. Larson Fredrik Bengzon, "The Finite Element Method: Theory, Implementation, and Applications", DOI 10.1007/978-3-642-33287-6, Springer Heidelberg New York Dordrecht London
- Hans Petter Langtangen ▪ Anders Logg, "Solving PDEs in Python The FEniCS Tutorial I", DOI 10.1007/978-3-319-52462-7, Simula SpringerBriefs on Computing
- Anders Logg ▪ Kent-Andre Mardal ▪ Garth N. Wells, "Automated Solution of Differential Equations by the Finite Element Method", DOI 10.1007/978-3-642-23099-8, 2011 The FEniCS Project.
- https://docs.fenicsproject.org/dolfinx/v0.9.0/python/demos.html
- https://jsdokken.com/dolfinx-tutorial/
- https://docs.djangoproject.com/en/5.1/

DRAFT